

Bus Reservation System

Professional Enterprise User Manual & Operations Guide

Document Assignment:	Semester Project Proposal Submission
Course Title:	Programming Fundamentals (Instructor: Muhammad Asad Ullah)
Academic Session:	2025 Session (2nd Semester)
Development Platform:	C++ Standard Console Architecture
Documentation Date:	June 2026

Development & Documentation Team

Student Name	Registration ID	Degree Program
Mahadi Ur Rehman	2510382	BS Business Information Technology (BS-BIT)
Rana Abdullah	2510386	BS Business Information Technology (BS-BIT)
Arshan Mirza	2510376	BS Computational Modeling and Data Analytics (BS-CMDA)

1. Executive Summary & Introduction

The Bus Reservation System is a modular, high-performance C++ console-based application engineered to simulate a modern, multi-tiered transit management platform. Designed with enterprise-level role segregation, the system provides distinct execution pathways for retail customers (User Panel), operations managers (Admin Panel), and financial auditors (Finance Panel).

The software leverages core structured programming principles, advanced file handling mechanics, deterministic array-based matrix mappings, and robust input sanitization to ensure high data integrity, secure transactional parsing, and real-time state persistence without relying on external relational database management systems.

2. Comprehensive System Architecture & Features

2.1 User Module

- **Account Lifecycle Management:** Secure, unique-constraint user registration alongside a multi-factor login mechanism equipped with a strict brute-force prevention threshold (maximum 3 consecutive authentication attempts).
- **Dynamic Inventory Queries:** Real-time route sorting and search modules optimized by multi-attribute parameters (alphabetic route filters, ascending/descending pricing tiers) using custom sorting algorithms.
- **Transactional Booking Engine:** Automated reservation system featuring an interactive, visual 2D matrix seat allocation map mapping open ([]) and occupied ([X]) assets.
- **Tokenized Proof-of-Purchase:** Deterministic cryptographic token generation used as a unique tracking identifier across the entire transactional lifecycle.
- **Feedback Loops:** Synchronous user review submission limits (300-character boundary) mapped directly into global system files.

2.2 Admin Module

- **Asset Management (CRUD):** Complete control over transit assets, enabling real-time addition, modification, soft deletion, and deep truncation of bus inventories.
- **Systemic Cascading Truncation ('Destroy Bus'):** Mission-critical operational route cancellation utility. Triggering a bus destruction permanently drops the asset from active registries and triggers an automated downstream batch creation of passenger financial refunds.
- **Security & Audit Logging:** Read-only access to time-stamped system activity logs tracking runtime events, account mutations, and structural system exceptions.

2.3 Finance Module

- **Granular Financial Reporting:** Computational engines that compile analytics for Monthly Gross Revenue, Approved Refund Attrition (Loss Tracking), and Consolidated Net Profit margins.

- **Isolated Claims Management:** Independent ledger management allowing financial officers to override, approve, or cancel pending cash distribution pipelines.

3. Engineering & Environment Specifications

3.1 System Requirements

- **Operating Systems:** Microsoft Windows 10/11, Linux (Ubuntu 20.04 LTS or later), or macOS Architecture.
- **Compilation Toolchain:** Any ISO-compliant C++ compiler supporting the standard library (G++ v9.0+, MinGW, MSVC, Clang).
- **Dependencies:** Standard Template Library (STL) components only. No external wrappers, third-party DLLs, or external packages required.

3.2 Compilation & Deployment Execution

To build and execute the application binary via a standard Command Line Interface (CLI), execute the following commands in your development directory:

```
# Step 1: Compile the source toolchain
g++ bus_reservation_system.cpp -o bus_system

# Step 2: Initialize active executable platform context
./bus_system
```

4. System Interface Protocols

4.1 Master Gateway Interface

Upon initialization, the runtime ecosystem directs all initialization sequences to the primary authentication gateway loop:

```
===Welcome To Bus Reservation System===
What would you like to do
0. Admin Log_in
1. Register Your Account
2. Log_in Your Account
3. Finance Log_in
4. Exit
```

4.2 Account Generation Protocols (User Signup)

1. Select Option 1 from the terminal gateway.
2. Provide a globally unique username. Valid characters are limited to standard alphanumeric strings (A-Z, a-z, 0-9) and underscores (_).

3. Passwords must pass standard enterprise structural validation metrics before database insertion:

- Minimum structural length of 8 characters.
- Inclusion of at least one uppercase alphabetic character (A-Z).
- Inclusion of at least one numerical digit (0-9).
- Inclusion of at least one non-alphanumeric special character (e.g., !, @, #, \$).

4.3 Active Sessions User Dashboard

Upon authentication, users are redirected to the management sub-shell layout:

```
===Bus Management===
1. Book Seat
2. Check Route Timing
3. Bus Details
4. Check My Booking
5. Cancel Booking
6. My All Bookings
7. Search Route
8. Change My Password
9. Review
10. My Refunds
11. Back To Login
```

5. Standard Operational Procedures (User Module)

5.1 Interactive Reservation Procedures

1. Enter Option 1 from the customer control loop.
2. Select desired sorting parameters to render available transit arrays.
3. Provide the targeted Bus Number when requested by the application interface context.
4. The application clears the stream buffer and renders a dynamic inventory map detailing active route status, metadata boundaries, and a physical seating representation matrix:
 - [] represents an unallocated, available asset.
 - [X] represents a locked, locked, or active booking asset.
5. Select an open numerical identifier. The system reserves the target address space and issues a persistent alphanumeric token string following the format: TKN-[username]-[busNo]-[seatNo].

CRITICAL SECURITY NOTE: This token is a hard cryptographic ledger key for your data profile. It must be retained to file cancellation requests, execute tracking inquiries, or claim processing refunds.

5.2 Booking Cancellation & Revocation Flows

1. Route to Option 5 ('Cancel Booking') within your terminal interface view.
2. Submit your unique booking token string to the active console input buffer.
3. The application parses seats.txt, verifies ownership rights against the currently active user session context, releases the targeted matrix address back to an available state ([]), and passes an execution flag to the activity logger.

6. Administrative Operations Manual

6.1 Admin Console Gateway

```
===Admin Menu===  
1. Add Bus  
2. Edit Bus  
3. Remove Bus  
4. Remove Account  
5. Change Password  
6. View Users  
7. View Bookings  
8. View Logs  
9. Destroy Bus  
10. View Refunds  
11. Approve Refund  
12. Cancel Refund  
13. Back To Login
```

6.2 Structural Route Demolition ('Destroy Bus')

When a route becomes completely unviable or encounters unexpected scheduling changes, admins can completely remove the asset via a high-priority demolition routine. This process executes the following steps programmatically:

- **Purge Target from Registry:** The target vehicle identifier is permanently stripped from buses.txt and logged into destroyed_buses.txt.
- **Scan Active Bookings File:** The application sweeps seats.txt to find all assigned dependencies linked to that vehicle.
- **Batch Generate Refunds:** All affected customer contracts are broken, structural records are purged, and a corresponding entry is pushed to refunds.txt flagged as PENDING.

7. Financial Control & Ledger Auditing

7.1 Initialization Vector

Selecting Main Menu Option 3 invokes the auditing subsystem. If the local initialization script fails to locate finance.txt within the root execution folder, the platform dynamically generates the authentication profile using secure system parameters:

- System Default Username: finance
- System Default Password: Finance@1234

7.2 Reporting Framework Specifications

- **Monthly Gross Revenue Ledger:** Iterates through every record in the ledger files, grouping transaction counts, aggregating gross pricing totals, and calculating metrics to isolate top-performing routes.
- **Claims Loss Review:** Evaluates the economic impact of refunds. This report compiles volume counts and totals exclusively from claims flagged as APPROVED.
- **Consolidated Net Profit Engine:** Runs real-time balance calculations by comparing total gross revenue against approved refund distributions:
Net Profit = Total Gross Revenue - Total Approved Refund Loss

8. Relational File Architecture

To maintain performance, the system uses an isolated, flat-file structure with data files acting as independent data tables. Transactions are managed through consistent parsing, read/write loops, and file stream handling:

System File Name	System Access Type	Core Data Schema & Storage Architecture
accounts.txt	Read / Write	Persists master registration profiles (User credentials).
buses.txt	Read / Write	Maintains records for active transit routes, schedules, and vehicle assets.
seats.txt	Read / Write	Tracks real-time spatial reservations using explicit seat coordinates.
activity_log.txt	Append Only	Provides a sequential audit trail with system-wide millisecond timestamps.
reviews.txt	Read / Write	Stores customer feedback logs up to a 300-character limit.
admin_log.txt	Static Read	Restricts administrative entry paths via predefined credential verification.

System File Name	System Access Type	Core Data Schema & Storage Architecture
destroyed_buses.txt	Append Only	Archives decommissioned route numbers to prevent data entry collisions.
refunds.txt	Read / Write	Central ledger managing all customer reimbursement workflows.
finance.txt	Read / Write	Manages access profiles for financial auditing personnel.

9. System Validation Rules & Security Matrix

Targeting Data Field	Programmatic Structural Validation Rules
Username	Restricted to standard alphanumeric characters and underscores (_). Space allocation forms are rejected.
Password Rule Matrix	Length >= 8 characters. Requires at least 1 uppercase letter, 1 digit, and 1 special symbol.
Bus Identifier	Limited to alphanumeric strings and hyphens (-). Space inputs are blocked.
Transit Route	Alpha strings separated exclusively by single directional hyphens (e.g., Lahore-Multan).
Seat Map Coordinate	Integer values bound strictly within the range of 1 <= Target Seat <= Max Capacity.
Pricing Vector	Positive integer values only. Floating points or negative signs reject the transaction.
Temporal Format	Standard 24-hour timestamp strings with a minimum structural footprint of 5 characters (e.g., 08:00AM).
Systemic Token	Immutable alphanumeric string containing no blank spaces.

10. Robust Exception Handling & Operational Safeguards

- Console Input Sanitization:** When numerical input fields receive unexpected character streams (such as entering alphanumeric text into a menu option), the system flags the error, purges the std::cin stream buffer, and resets the interface state.

- **I/O Stream Assurance:** If a data file is missing or locked during access, file system exceptions intercept the fault. The application creates missing ledger dependencies on the fly or halts the affected thread without crashing the main application process.
- **State Conflict Prevention:** The validation layer blocks operations that target already processed data, such as trying to double-approve or re-cancel an archived refund token.

11. Core Computational Implementations

- **Low-Level File I/O Management:** The storage layer relies on structured file streams (`std::ifstream`, `std::ofstream`, `std::fstream`) to handle reading, parsing, and updating data records across system restarts.
- **Structured Dynamic Entities:** Key system entities like Bus, User, and Booking records are organized using native data structures (`struct`), keeping related datasets clean and well-encapsulated.
- **Modular Architecture:** Responsibilities are split cleanly into decoupled functions, keeping business logic distinct from terminal rendering routines.
- **Array Manipulation & Matrix Mapping:** Seating configurations are managed via internal array mappings, tracking physical inventory states by direct memory references.
- **Sorting Performance:** The catalog system uses an optimized Bubble Sort algorithm, enabling on-the-fly ordering of available routes by price point, route code, or bus number.